

Manual do Programador - FitTracker	2
Índice	2
1. Introdução.....	3
2. Arquitetura da Aplicação.....	3
3. Ambiente de Desenvolvimento	4
3.1 Requisitos de Sistema.....	4
3.2 Configuração do Ambiente	4
3.3 Estrutura de Diretórios.....	5
4. Base de Dados.....	5
4.1 Modelo de Dados	5
4.2 Scripts de Criação.....	6
4.3 Acesso a Dados.....	7
5. Componentes Principais	8
5.1 Sistema de Autenticação	8
5.2 Gestão de Treinos	8
5.3 Gestão de Objetivos.....	9
5.4 Visualização de Dados.....	9
5.5 Perfil de Utilizador.....	10
6. Interface de Utilizador	10
6.1 Formulários Principais	10
6.2 Controlos Personalizados	10
6.3 Navegação entre Formulários.....	11
7. Padrões de Código.....	11
7.1 Gestão de Erros.....	11
7.2 Boas Práticas.....	11
8. Extensão da Aplicação	12
8.1 Adicionar Novos Tipos de Treino	12
8.2 Implementar Novas Funcionalidades	12
8.3 Personalizar a Interface.....	13
10. Implantação	14
10.1 Criação de Instalador	14
10.2 Requisitos de Sistema para Utilizadores Finais.....	14

Manual do Programador - FitTracker

Versão 1.0

Última atualização: 17 de junho de 2025

Índice

1. Introdução
 2. Arquitetura da Aplicação
 3. Ambiente de Desenvolvimento
 - Requisitos de Sistema
 - Configuração do Ambiente
 - Estrutura de Diretórios
 4. Base de Dados
 - Modelo de Dados
 - Scripts de Criação
 - Acesso a Dados
 5. Componentes Principais
 - Sistema de Autenticação
 - Gestão de Treinos
 - Gestão de Objetivos
 - Visualização de Dados
 - Perfil de Utilizador
 6. Interface de Utilizador
 - Formulários Principais
 - Controlos Personalizados
 - Navegação entre Formulários
 7. Padrões de Código
 - Gestão de Erros
 - Boas Práticas
 8. Extensão da Aplicação
 - Adicionar Novos Tipos de Treino
 - Implementar Novas Funcionalidades
 - Personalizar a Interface
 9. Implantação
 - Criação de Instalador
 - Requisitos de Sistema para Utilizadores Finais
-

1. Introdução

Este manual do programador fornece informações técnicas detalhadas sobre a aplicação **FitTracker**, desenvolvida em C# Windows Forms. O documento destina-se a programadores.

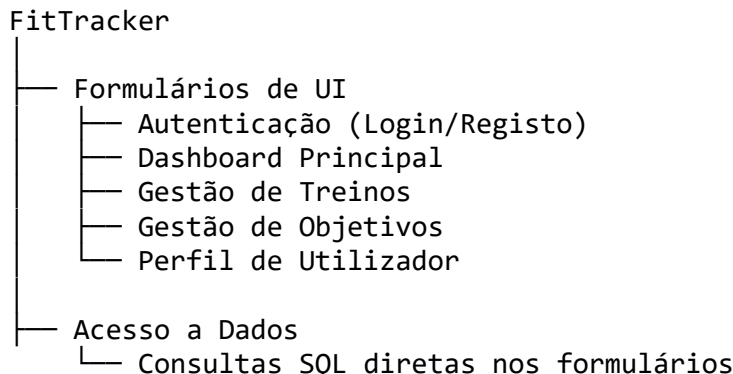
A FitTracker é uma aplicação de gestão de atividades físicas que permite aos utilizadores registar treinos, definir objetivos e visualizar estatísticas do seu progresso. A aplicação utiliza uma base de dados SQL Server LocalDB para armazenamento de dados e a biblioteca Guna UI2 para elementos de interface modernos.

Visão Geral da Aplicação

Visão Geral da Aplicação

2. Arquitetura da Aplicação

A aplicação segue uma arquitetura tradicional de Windows Forms, com formulários separados para cada funcionalidade principal. A estrutura geral pode ser descrita da seguinte forma:



A aplicação não segue estritamente o padrão MVC (Model-View-Controller) ou outros padrões de design formais. Em vez disso, a lógica e o acesso a dados estão integrados diretamente nos formulários, o que é comum em aplicações Windows Forms mais simples.

Diagrama de Arquitetura

Diagrama de Arquitetura

3. Ambiente de Desenvolvimento

3.1 Requisitos de Sistema

Para desenvolver a aplicação FitTracker, é necessário:

- **Sistema Operativo:** Windows 10 ou superior
- **IDE:** Visual Studio 2019 ou superior
- **Framework:** .NET Framework 4.7.2 ou superior
- **Base de Dados:** SQL Server LocalDB (incluído no Visual Studio)
- **Bibliotecas Adicionais:**
 - Guna UI2 WinForms (para controlos modernos)
 - System.Data.SqlClient (para acesso a dados)
 - System.Windows.Forms.DataVisualization (para gráficos)

3.2 Configuração do Ambiente

1. Instalação do Visual Studio:

- Instale o Visual Studio com o workload “Desenvolvimento para Desktop com .NET”
- Certifique-se de que o SQL Server LocalDB está incluído na instalação

Instalação do Visual Studio

Instalação do Visual Studio

2. Configuração da Base de Dados:

- Execute o SQL Server Management Studio (SSMS)
- Conecte-se à instância LocalDB: (localdb)\MSSQLLocalDB
- Crie uma nova base de dados chamada “FitTracker”
- Execute os scripts de criação de tabelas (fornecidos na secção 4.2)

3. Instalação da Biblioteca Guna UI2:

- No Visual Studio, aceda a Ferramentas > Gestor de Pacotes NuGet > Gerir Pacotes NuGet para a Solução
- Procure por “Guna.UI2.WinForms” e instale o pacote

Instalação do Guna UI2

Instalação do Guna UI2

4. Configuração da String de Conexão:

- A string de conexão utilizada na aplicação é:
"Server=(localdb)\MSSQLLocalDB;Database=FitTracker;Trusted_Connection=True"
- Esta string está definida diretamente no código; para uma aplicação em produção, considere movê-la para um arquivo de configuração

3.3 Estrutura de Diretórios

A estrutura de diretórios do projeto é organizada da seguinte forma:

```
FitTracker/
├── Properties/                # Configurações do projeto
├── Forms/                    # Formulários da aplicação
│   ├── teste_login.cs        # Formulário de login
│   ├── CriarNovaContaNova.cs # Formulário de registo
│   ├── main.cs               # Dashboard principal
│   ├── adicionarTreino.cs    # Adicionar treino
│   ├── EditarTreino.cs       # Editar treino
│   ├── RemoverTreino.cs      # Remover treino
│   ├── Objetivos.cs          # Gestão de objetivos
│   └── perfil.cs             # Perfil de utilizador
└── Program.cs                # Ponto de entrada da aplicação
```

Estrutura de Diretórios

Estrutura de Diretórios

4. Base de Dados

4.1 Modelo de Dados

A base de dados FitTracker consiste nas seguintes tabelas principais:

1. **Utilizadores:** Armazena informações de conta dos utilizadores
 - Id (PK, int, auto-incremento)
 - Nome (varchar(50), único)
 - PalavraPasse (varchar(50))
2. **TipoTreino:** Catálogo de tipos de treino disponíveis
 - Id (PK, int, auto-incremento)
 - Nome (varchar(50))
3. **Treinos:** Registo de treinos realizados
 - Id (PK, int, auto-incremento)
 - IdUtilizador (FK -> Utilizadores.Id)
 - IdTipoTreino (FK -> TipoTreino.Id)
 - Data (date)
 - Duracao (int) - em minutos
 - Notas (varchar(255), nullable)
4. **Objetivos:** Objetivos de treino definidos pelos utilizadores

- Id (PK, int, auto-incremento)
 - IdUtilizador (FK -> Utilizadores.Id)
 - TipoTreinoDesejado (FK -> TipoTreino.Id, nullable)
 - DuracaoMeta (int) - em minutos
 - DataLimite (date)
 - Notas (varchar(255), nullable)
5. **vTreinosComNomeTipo**: Vista que combina informações de treinos com nomes dos tipos de treino

Diagrama ER

Diagrama ER

4.2 Scripts de Criação

Abaixo estão os scripts SQL para criar as tabelas necessárias:

-- Tabela de Utilizadores

```
CREATE TABLE Utilizadores (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Nome VARCHAR(50) UNIQUE NOT NULL,
    PalavraPasse VARCHAR(50) NOT NULL
);
```

-- Tabela de Tipos de Treino

```
CREATE TABLE TipoTreino (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Nome VARCHAR(50) NOT NULL
);
```

-- Inserir tipos de treino padrão

```
INSERT INTO TipoTreino (Nome) VALUES
('Corrida'), ('Musculação'), ('Natação'), ('Ciclismo'),
('Yoga'), ('Pilates'), ('Caminhada'), ('Futebol'), ('Basquetebol');
```

-- Tabela de Treinos

```
CREATE TABLE Treinos (
    Id INT PRIMARY KEY IDENTITY(1,1),
    IdUtilizador INT NOT NULL,
    IdTipoTreino INT NOT NULL,
    Data DATE NOT NULL,
    Duracao INT NOT NULL,
    Notas VARCHAR(255),
    FOREIGN KEY (IdUtilizador) REFERENCES Utilizadores(Id),
    FOREIGN KEY (IdTipoTreino) REFERENCES TipoTreino(Id)
);
```

-- Tabela de Objetivos

```
CREATE TABLE Objetivos (
    Id INT PRIMARY KEY IDENTITY(1,1),
```

```

    IdUtilizador INT NOT NULL,
    TipoTreinoDesejado INT,
    DuracaoMeta INT NOT NULL,
    DataLimite DATE NOT NULL,
    Notas VARCHAR(255),
    FOREIGN KEY (IdUtilizador) REFERENCES Utilizadores(Id),
    FOREIGN KEY (TipoTreinoDesejado) REFERENCES TipoTreino(Id)
);

```

```

-- Vista para combinar treinos com nomes de tipos
CREATE VIEW vTreinosComNomeTipo AS
SELECT
    T.Id, T.IdUtilizador, T.Data, T.Duracao, T.Notas,
    TT.Id AS IdTipoTreino, TT.Nome AS TipoTreino
FROM
    Treinos T
    INNER JOIN TipoTreino TT ON T.IdTipoTreino = TT.Id;

```

4.3 Acesso a Dados

A aplicação utiliza ADO.NET para acesso a dados, com consultas SQL diretas incorporadas no código dos formulários. Abaixo está um exemplo de como o acesso a dados é implementado:

```

// Exemplo de método para obter treinos de um utilizador
private void CarregarTreinos()
{
    string connString =
        "Server=(localdb)\\MSSQLLocalDB;Database=FitTracker;Trusted_Connection=Tr
ue";
    using (SqlConnection conn = new SqlConnection(connString))
    {
        conn.Open();

        string query = @"
            SELECT T.Id AS Id, T.Data, TT.Nome AS TipoDeTreino,
            T.Duracao, T.Notas
            FROM Treinos T
            INNER JOIN TipoTreino TT ON T.IdTipoTreino = TT.Id
            WHERE T.IdUtilizador = @idUtilizador
            ORDER BY T.Data DESC";

        using (SqlCommand cmd = new SqlCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@idUtilizador",
SessaoUtilizador.Id);
            SqlDataAdapter da = new SqlDataAdapter(cmd);
            DataTable dt = new DataTable();
            da.Fill(dt);
            dataGridView1.DataSource = dt;

```

```

        dataGridView1.AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fill;
    }
}

```

Exemplo de Acesso a Dados

Exemplo de Acesso a Dados

5. Componentes Principais

5.1 Sistema de Autenticação

O sistema de autenticação é composto por dois formulários principais:

1. **teste_login.cs**: Responsável pela autenticação de utilizadores existentes
2. **CriarNovaContaNova.cs**: Responsável pelo registo de novos utilizadores

A autenticação é realizada através de consultas diretas à tabela Utilizadores, verificando se o nome de utilizador e a palavra-passe correspondem a um registo existente.

Após o login bem-sucedido, as informações do utilizador são armazenadas numa classe estática chamada SessaoUtilizador, que mantém o ID e o nome do utilizador durante toda a sessão da aplicação.

```

// Classe SessaoUtilizador (não mostrada no código fornecido, mas
inferida)
public static class SessaoUtilizador
{
    public static int Id { get; set; }
    public static string Nome { get; set; }
}

```

Fluxo de Autenticação

Fluxo de Autenticação

5.2 Gestão de Treinos

A gestão de treinos é dividida em três formulários:

1. **adicionarTreino.cs**: Permite adicionar novos treinos
2. **EditarTreino.cs**: Permite editar treinos existentes
3. **Removertreino.cs**: Permite remover treinos existentes

Cada formulário interage diretamente com a tabela Treinos através de consultas SQL para inserir, atualizar ou excluir registros. Os formulários também incluem validação básica para garantir que todos os campos obrigatórios sejam preenchidos.

Gestão de Treinos

Gestão de Treinos

5.3 Gestão de Objetivos

A gestão de objetivos é implementada no formulário **Objetivos.cs**, que permite aos utilizadores:

- Adicionar novos objetivos
- Editar objetivos existentes
- Remover objetivos
- Visualizar o progresso em relação aos objetivos

O formulário inclui uma consulta SQL complexa que calcula o progresso atual em relação a cada objetivo, somando a duração dos treinos relevantes até à data limite.

Gestão de Objetivos

Gestão de Objetivos

5.4 Visualização de Dados

A visualização de dados é implementada principalmente no formulário **main.cs**, que inclui:

1. Uma tabela (DataGridView) mostrando os treinos recentes
2. Um gráfico de barras mostrando a atividade por dia da semana

O gráfico é criado utilizando o componente Chart do namespace System.Windows.Forms.DataVisualization.Charting, com dados obtidos através de uma consulta SQL que agrupa os treinos por dia da semana.

```
private void CarregarGraficoSemana()
{
    // Consulta SQL para obter minutos de treino por dia da semana
    string query = @"
        SELECT
            DATEPART(WEEKDAY, Data) AS DiaNum,
            SUM(Duracao) AS TotalMinutos
        FROM Treinos
        WHERE IdUtilizador = @id
        GROUP BY DATEPART(WEEKDAY, Data);";

    // Código para preencher o gráfico com os dados obtidos
    // ...
}
```

Visualização de Dados

Visualização de Dados

5.5 Perfil de Utilizador

O perfil de utilizador é gerido no formulário **perfil.cs**, que permite aos utilizadores:

- Visualizar as suas informações de conta
- Editar as suas informações (através do formulário EditarPerfil, não incluído no código fornecido)
- Atualizar a visualização das informações

Perfil de Utilizador

Perfil de Utilizador

6. Interface de Utilizador

6.1 Formulários Principais

A aplicação é composta pelos seguintes formulários principais:

1. **teste_login.cs**: Ecrã de login
2. **CriarNovaContaNova.cs**: Ecrã de registo
3. **main.cs**: Dashboard principal
4. **adicionarTreino.cs**: Adicionar treino
5. **EditarTreino.cs**: Editar treino
6. **Removertreino.cs**: Remover treino
7. **Objetivos.cs**: Gestão de objetivos
8. **perfil.cs**: Perfil de utilizador

Cada formulário é uma classe que herda de Form e inclui componentes de UI, eventos e lógica de negócios.

Hierarquia de Formulários

Hierarquia de Formulários

6.2 Controlos Personalizados

A aplicação utiliza controlos padrão do Windows Forms e controlos personalizados da biblioteca Guna UI2, incluindo:

- **Guna2Button**: Botões modernos com efeitos visuais
- **Guna2TextBox**: Campos de texto com aparência moderna
- **Guna2DateTimePicker**: Seletor de data com aparência moderna
- **Guna2CheckBox**: Caixas de verificação com aparência moderna

- **Guna2NumericUpDown:** Controlo numérico com aparência moderna
- Controlos Guna UI2

Controlos Guna UI2

6.3 Navegação entre Formulários

A navegação entre formulários é implementada através da criação de novas instâncias de formulários e da ocultação ou fecho do formulário atual:

```
private void button1_Click(object sender, EventArgs e)
{
    var Home = new main();
    Home.Show();
    this.Hide(); // ou this.Close();
}
```

Este padrão é consistente em toda a aplicação.

Fluxo de Navegação

Fluxo de Navegação

7. Padrões de Código

7.1 Gestão de Erros

A gestão de erros na aplicação é básica, consistindo principalmente em:

- Verificações de campos vazios antes de operações de base de dados
- Mensagens de erro exibidas através de `MessageBox.Show()`
- Sem tratamento estruturado de exceções (try-catch) na maioria das operações

Exemplo de validação básica:

```
if (string.IsNullOrEmpty(nome) || string.IsNullOrEmpty(password))
{
    MessageBox.Show("Preencha todos os campos.");
    return;
}
```

Exemplo de Gestão de Erros

Exemplo de Gestão de Erros

7.2 Boas Práticas

Algumas boas práticas observadas no código:

- Utilização de parâmetros SQL para prevenir injeção SQL

- Liberação adequada de recursos através de blocos using
- Separação básica de responsabilidades em métodos distintos

Áreas para melhoria:

- Implementar tratamento estruturado de exceções
 - Centralizar o acesso a dados em classes dedicadas
 - Implementar validação mais robusta de entrada de utilizador
 - Melhorar a segurança das palavras-passe (hashing)
-

8. Extensão da Aplicação

8.1 Adicionar Novos Tipos de Treino

Para adicionar novos tipos de treino:

1. Insira novos registos na tabela TipoTreino:
`INSERT INTO TipoTreino (Nome) VALUES ('Novo Tipo de Treino');`
2. Os novos tipos serão automaticamente carregados nos ComboBox relevantes, pois estes são preenchidos dinamicamente a partir da tabela TipoTreino.

Adicionar Tipos de Treino

Adicionar Tipos de Treino

8.2 Implementar Novas Funcionalidades

Para implementar novas funcionalidades, siga estas etapas gerais:

1. **Criar um novo formulário:**
 - No Visual Studio, clique com o botão direito no projeto > Adicionar > Windows Form
 - Desenhe a interface do utilizador
 - Implemente a lógica necessária
2. **Adicionar navegação para o novo formulário:**
 - Adicione um botão na barra lateral ou em outro local apropriado
 - Implemente o evento de clique para abrir o novo formulário:

```
private void btnNovaFuncionalidade_Click(object sender, EventArgs e)
{
    var novoFormulario = new NovaFuncionalidade();
    novoFormulario.Show();
    this.Hide();
}
```

3. **Atualizar a base de dados (se necessário):**
 - Crie novas tabelas ou modifique as existentes
 - Atualize as consultas SQL relevantes

Implementar Novas Funcionalidades

Implementar Novas Funcionalidades

8.3 Personalizar a Interface

Para personalizar a interface da aplicação:

1. **Modificar o esquema de cores:**

- Altere as propriedades BackColor e ForeColor dos controlos
- Para uma abordagem mais sistemática, crie uma classe de estilos:

```
public static class EstilosUI
{
    public static Color CorPrimaria = Color.FromArgb(65, 105, 225);
    public static Color CorSecundaria = Color.FromArgb(240, 240, 240);

    public static void AplicarEstiloBotao(Button botao)
    {
        botao.BackColor = CorPrimaria;
        botao.ForeColor = Color.White;
        botao.FlatStyle = FlatStyle.Flat;
        botao.FlatAppearance.BorderSize = 0;
    }

    // Outros métodos de estilo...
}
```

2. **Melhorar o layout:**

- Utilize TableLayoutPanel ou FlowLayoutPanel para melhor organização
- Defina margens e espaçamento consistentes

3. **Adicionar ícones e imagens:**

- Adicione recursos ao projeto (clique com o botão direito no projeto > Adicionar > Novo Item > Recursos)
- Utilize os recursos nos controlos apropriados

Personalizar Interface

Personalizar Interface

```
// Interagir com a interface
TextBox txtNome = janela.Get<TextBox>("TextBox1");
TextBox txtPassword = janela.Get<TextBox>("TextBox2");
Button btnLogin = janela.Get<Button>("button2");

txtNome.Text = "utilizador_teste";
txtPassword.Text = "senha_teste";
btnLogin.Click();
```

```

        // Verificar resultado
        Window janelaMain = app.GetWindow("main");
        Assert.IsNotNull(janelaMain);

        // Limpar
        app.Close();
    }

```

Testes de Interface

Testes de Interface

10. Implantação

10.1 Criação de Instalador

Para criar um instalador para a aplicação:

1. Adicione um projeto de instalação ao solution:
 - Clique com o botão direito na solution > Adicionar > Novo Projeto
 - Selecione “Projeto de Instalação”
2. Configure o projeto de instalação:
 - Adicione os ficheiros de saída do projeto principal
 - Configure pré-requisitos (.NET Framework, SQL Server LocalDB)
 - Defina atalhos e associações de ficheiros
3. Compile o projeto de instalação para gerar um ficheiro MSI

Criação de Instalador

Criação de Instalador

10.2 Requisitos de Sistema para Utilizadores Finais

Os requisitos mínimos para executar a aplicação são:

Componente	Requisito Mínimo
Sistema Operativo	Windows 7 SP1 ou superior
Framework	.NET Framework 4.7.2 ou superior
Base de Dados	SQL Server LocalDB 2016 ou superior
Espaço em Disco	100 MB para a aplicação + 200 MB para a base de dados
Memória RAM	2 GB ou superior
Resolução de Ecrã	1024x768 ou superior
Requisitos de Sistema	

Requisitos de Sistema

© 2025 FitTracker. Todos os direitos reservados.